

Introduction to Software Testing

Course: 6.102 Software Construction

Topic: Testing

Core Goals:

- Safe from bugs (SFB) 1
- Easy to understand (ETU) 1
- Ready for change (RFC) 1

Slide 2: Learning Objectives

By the end of this lecture, you should:

- Understand the value of testing and the test-first programming process 1.
- Be able to judge a test suite for correctness, thoroughness, and size 1.
- Design test suites by partitioning input spaces and selecting boundary values 1.
- Measure test effectiveness via code coverage 1.
- Distinguish between black box vs. glass box, unit vs. integration, and automated regression testing 1.

Slide 3: Validation

Validation: The general process of uncovering problems and increasing confidence in correctness 2.

Three main approaches:

- **Formal Reasoning (Verification):** Constructing formal proofs of correctness 2.
- **Code Review:** Having others read and reason about your code 2.
- **Testing:** Running the program on selected inputs and checking results 2.

Slide 4: Verification vs. Testing

- Verification is often tedious to do by hand; automated tool support is still an active research area 2.
- It is used for small, high-stakes components like OS schedulers or filesystems 2.
- Testing is the most common validation technique in industry, relying on execution rather than proof 2.

Slide 5: Why Software Testing is Hard (1/2)

- **Exhaustive testing is infeasible.**
 - Example: A 32-bit floating-point multiply ($a \times b$) has 2^{64} test cases 3.
- **Haphazard testing ("just try it") is unreliable.**
 - Unless the program is extremely buggy, arbitrary inputs are unlikely to find specific flaws 3.

Slide 6: Why Software Testing is Hard (2/2)

- **Software is not "continuous."**
- In physical engineering, systems often show "cracks" before failure or follow a uniform distribution 4, 5.
- In software, behavior varies discontinuously and discretely 5.
- A program may work for billions of inputs and fail at a single boundary point 5.

Slide 7: Case Study: Discontinuity in Action

- **The Pentium Division Bug:** Affected 1 in 9 billion divisions 5.
- **Ariane 5 Rocket Failure (\$1 Billion cost):**
 - Occurred during a 64-bit float to 16-bit integer conversion 6.
 - The value overflowed, an exception was thrown, but the handler was disabled for efficiency 6.
 - The software crashed, causing the rocket to self-destruct 6.

Slide 8: Core Definitions

- **Module:** A part of a system that can be designed, implemented, and tested separately (e.g., a function) 7.
- **Specification (Spec):** Describes the behavior of a module, including parameter types, return values, and constraints 8.
- **Implementation:** The actual code (the body) providing the behavior 8.
- **Client:** The code that calls the module 8.

Slide 9: Test Cases and Suites

- **Test Case:** A specific choice of inputs paired with the expected output behavior as required by the spec 9.
- **Test Suite:** A collection of test cases for a specific module 9.
- Effective testing requires choosing these cases systematically rather than randomly 10.

Slide 10: Test-First Programming

Develop a single function in this specific order:

1. **Spec:** Write the specification 11.
2. **Test:** Write tests that exercise the specification 11.
3. **Implement:** Write the code 11.

Slide 11: Benefits of Test-First Programming

- **Safety from Bugs:** Don't leave testing until the end when you have a "big pile of unvalidated code" 11.
- **Easier Debugging:** When you test as you develop, you know the bug is likely in the small piece of code you just wrote 11.
- **Validation of Spec:** Writing tests helps you find ambiguities or missing corner cases in the specification before you waste time implementing them 14.

Slide 12: Goals of Systematic Testing

A good test suite has three properties:

- **Correct:** It is a legal client of the spec and accepts all legal implementations 15.
- **Thorough:** It finds bugs that programmers are likely to make 15.
- **Small:** It is fast to run and easy to maintain 12.

Slide 13: Choosing Test Cases by Partitioning

- Divide the input space into subdomains 16.
- Each subdomain consists of a set of "similar" inputs where the program is expected to behave similarly 17.
- Pick one test case from each subdomain to form the test suite 16.

Slide 14: Mathematical Partitioning

A valid partition must be:

- **Disjoint:** Subdomains do not overlap 18.
- **Complete:** The union of subdomains covers the entire legal input space 18.
- **Nonempty:** You must be able to choose at least one test case from each 18.

Slide 15: Example: `Math.abs(a: number)`

- **Input Space:** All numbers.
- **Logic:** $\text{abs}(a)$ stays a if $a \geq 0$, and becomes $-a$ if $a < 0$ 19.
- **Partition:**
 - $a \geq 0$
 - $a < 0$ 20
- **Test Cases:** $a = 17$, $a = -3$ 20.

Slide 16: Example: `Math.max(a, b)`

- **Input Space:** Two-dimensional (pairs of a, b).
- **Partition:**
 - $a < b$
 - $a > b$
 - $a = b$ 18, 21
- **Test Cases:** (1, 2), (10, -8), (9, 9) 18.

Slide 17: Include Boundaries

Bugs often occur at boundaries between subdomains 22.

Why?

- Off-by-one errors (using $\leq$ instead of $<$) 22.
- Special cases in code 22.
- Discontinuities (e.g., numeric overflow) 22.

Slide 18: Common Boundaries to Test

- **Numbers:** 0, maximum/minimum values (e.g., `Number.MAX_SAFE_INTEGER`) 22, 23.
- **Collections:** Empty string, empty array, empty set 22.
- **Sequences:** The first and last elements of an array or string 22.

Slide 19: Refining the abs() Partition

Instead of just two subdomains, we incorporate 0 as its own subdomain:

- $a < 0$
- $a = 0$
- $a > 0 \text{ } 24$

Test Suite: $a = -3, a = 0, a = 17 \text{ } 24$.

Slide 20: Case Study: BigInt Multiplication

- **Inputs:** two BigInt values (a, b) 25.
- **Possible Concerns:** Signs, Magnitude (fits in number vs. too large), Boundaries (0, 1) 23, 25.
- **Partitioning a and b independently:**
 - 0
 - 1
 - Small positive integer
 - Small negative integer
 - Large positive integer
 - Large negative integer 26.

Slide 21: The Combinatorial Explosion

- If we take the Cartesian Product of the 6 subdomains for a and 6 for b , we get $6 \times 6 = 36$ subdomains 26, 27.
- For n parameters, this grows exponentially 27.
- **Solution:** Treat features as separate partitions 28. You can cover multiple subdomains from different partitions with a single test case 29, 30.

Slide 22: Automated Unit Testing

- **Unit Test:** Tests an individual module in isolation 31.
- **Automation:** Running tests and checking results without manual intervention 31.
- **Test Driver:** Code that invokes the module and checks results (e.g., Mocha for TypeScript) 31, 32.

Slide 23: Mocha Basics

- `describe()`: Groups related tests (a test suite) 33.
- `it()`: Defines a single test case 32.
- **Assertions:** Functions that check if the actual result matches the expected result 32.

```
it("covers a < b", function() {  
    assert.strictEqual(Math.max(1, 2), 2);  
});
```

Slide 24: Writing Effective Assertions

- **Order matters:** `assert.strictEqual(actual, expected)` 34.
- **Standard types:** `strictEqual` works for strings and numbers 35.
- **Data structures (Arrays, Sets):** `strictEqual` fails because it checks identity, not content 35, 36.
- Use `assert.deepStrictEqual()` for built-in arrays/sets 36.
- Or check individual properties (e.g., `assert.strictEqual(set.size, 1)`) 37.

Slide 25: Documenting Strategy

- You must document your testing strategy so it is visible to readers 38.
- Place the strategy in a comment inside `describe()` 38.
- List the partitions and subdomains 38.
- Name your `it()` blocks based on the subdomains they cover 39.

Slide 26: Black Box vs. Glass Box Testing

- **Black Box Testing:** Choosing test cases only from the spec. You don't look at the code 40.
- **Glass Box Testing:** Choosing test cases with knowledge of the implementation 40.
- **Example:** If the code uses different algorithms for small vs. large arrays, partition at that specific threshold 40, 41.

Slide 27: Code Coverage

How thoroughly does your test suite exercise the program? 42

- **Statement Coverage:** Is every statement run? 42
- **Branch Coverage:** Is every if/else direction taken? 42
- **Path Coverage:** Is every possible combination of branches taken? (Usually infeasible) 42, 43.

Note: 100% statement coverage is a common goal but does not guarantee the absence of bugs 43.

Slide 28: Using Coverage Tools

Tools like `c8` measure coverage automatically 44.

Process:

1. Run black box tests.
2. Check coverage report.
3. Add glass box tests to cover "red" (unexecuted) lines 44, 45.

Slide 29: Unit vs. Integration Testing

- **Unit Testing:** Isolated. If it fails, you know exactly where the bug is 46.
- **Integration Testing:** Tests combinations of modules 46.
- **Risk:** If you only have integration tests, finding a bug is much harder because it could be anywhere in the system 46.

Slide 30: Isolation and Stubs

- To truly isolate a unit test, avoid calling other complex modules 47.
- **Stub:** A "mock" version of a module that returns fixed values instead of performing real logic 48.
- **Example:** A stub for `load()` might return a hardcoded string instead of reading from the disk 48.

Slide 31: Automated Regression Testing

- **Regression:** Reintroducing an old bug while making changes 49.
- **Regression Testing:** Running all tests after every change to ensure baseline behavior is preserved 49.
- **Rule:** When you find a bug, write a test case for it immediately (Test-first debugging) 50, 51.

Slide 32: Iterative Development

Testing and implementation are iterative, not linear 52.

Plan for iteration:

1. Start with a simple spec and a few partitions 53.
2. Implement a "brute-force" version first to validate the tests 54.
3. Refine and improve steadily 54.

Slide 33: Summary

- **Test-first programming:** Spec → Test → Implement 55.
- **Systematic Testing:** Use partitions and boundaries for a thorough, small suite 55.
- **Automation:** Use Mocha and coverage tools 55.
- **Regression:** Run tests frequently to keep software Safe from Bugs 56.